Week 19 Homework: Analyze Time and Generate Research Ideas

Goal: evaluate a small time-series model honestly, then use Week 18 evidence to choose a testable primary research idea and a credible backup.

Estimated effort. About 75–90 minutes for Part I and 90–120 minutes for Part II. Stop and record the obstacle if a technical task takes more than 20 minutes without progress; explaining a failed attempt is better than hiding it.

Deliverables. Submit (1) `week19_timeseries.py`, (2) a time plot and a test-period prediction plot, (3) a three-model error table and residual interpretation, (4) six research idea cards, (5) an evidence-based shortlist, (6) two reality checks, (7) Proposal Seed v0.2 with a primary and backup idea, (8) a 72-hour action and decision rule, and (9) the Feynman Technique post.

AI policy. Part I, Steps 1–4 are the weekly **manual programming core**: do not use AI to fill any `TODO`, write the metric functions, choose code, or debug those steps. Consult Python, NumPy, pandas, Matplotlib, and scikit-learn documentation. After your code runs, AI may help explain an error or critique your written interpretation, but you must keep your original answer and record what changed. In Part II, first generate all six ideas yourself. AI may then challenge assumptions, suggest search terms, or ask skeptical questions; it may not choose the topic or invent evidence. Disclose any AI use and personally verify every factual claim and source.

Part I: Time-Series Analysis and Modeling

Research question for the exercise

For one synthetic building, can a linear model using only information available at the end of hour $t-1$ predict hour t load better than (a) repeating the last observed value and (b) repeating the value from the same hour yesterday?

The data are synthetic so that every student can run the exercise. A successful result demonstrates the evaluation process; it does not establish how a real building behaves.

Step 1: Create and audit the series—no AI

Type the starter code into `week19_timeseries.py`. Complete every `TODO` by consulting official documentation.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(19)
n_hours = 24 * 60
time = pd.date_range("2026-01-01", periods=n_hours, freq="h")
t = np.arange(n_hours)
hour = time.hour.to_numpy()
weekday = time.dayofweek.to_numpy()

daily = 7.0 * np.sin(2 * np.pi * (hour - 7) / 24)
weekend = np.where(weekday >= 5, -4.0, 0.0)
trend = 0.004 * t
noise = rng.normal(0, 1.8, n_hours)
load = 32.0 + daily + weekend + trend + noise

df = pd.DataFrame({"time": time, "load": load})

# Simulate a short meter communication failure.
df.loc[410:413, "load"] = np.nan

# TODO 1: sort rows by time and set time as the index.

# TODO 2: print the first timestamp, last timestamp, number of rows,
# number of missing load values, and whether the index is increasing.

# TODO 3: reindex to an hourly grid. Hint: use .asfreq("h").

# Preserve evidence that a value was originally missing.
df["was_missing"] = df["load"].isna().astype(int)

# TODO 4: fill only short internal gaps by time interpolation.
```

```
# TODO 5: plot the complete load series and save it as
# "week19_time_plot.png". Label both axes.
```

Add a comment above every completed `TODO` explaining the assumption behind the operation. Then answer:

1. What does one row represent? What are the target's units?
2. Which visible patterns appear daily, weekly, or gradually over time?
3. Why would replacing the four missing values with zero be misleading?
4. What assumption does time interpolation make during the missing interval?
5. Which patterns were deliberately built into the synthetic generator? Which visible variation remains random?

Step 2: Create features without seeing the future—no AI

Continue the file:

```
# Calendar features known at prediction time.
df["hour_sin"] = np.sin(2 * np.pi * df.index.hour / 24)
df["hour_cos"] = np.cos(2 * np.pi * df.index.hour / 24)
df["is_weekend"] = (df.index.dayofweek >= 5).astype(int)

# TODO 6: create lag_1 and lag_24 from load.

# TODO 7: create mean_24 using the previous 24 load values.
# Shift before rolling so the current target is excluded.

# TODO 8: remove rows made incomplete by lag/rolling operations.

features = [
    "lag_1", "lag_24", "mean_24",
    "hour_sin", "hour_cos", "is_weekend", "was_missing"
]
```

Answer:

1. At 3:00 p.m. on a Tuesday, what time does `lag_24` describe?
2. Why must `mean_24` use `shift(1)` before `rolling(24)`?
3. Why are sine and cosine better than treating hour 23 and hour 0 as far apart?
4. After dropping incomplete rows, how many rows remain and why were the earliest rows removed?

Step 3: Split chronologically and build baselines—no AI

Continue the file:

```
# TODO 9: set cut to 80% of the available rows.

# TODO 10: use .iloc to place the earlier 80% in train and
# the later 20% in test. Do not shuffle.

# TODO 11: create test predictions for these two baselines:
# persistence = lag_1
# seasonal_naive = lag_24

# TODO 12: print the final training timestamp and first test timestamp.
```

In two to four sentences, explain why a random train/test split would give a less honest forecast evaluation. State the exact test period created by your code.

Step 4: Fit one model and implement metrics—no AI

Continue the file. NumPy will solve the linear-regression least-squares problem, so no additional machine-learning package is required.

```
# TODO 13: create X_train by placing a column of ones before
# train[features]. The ones create the intercept term.

# TODO 14: use np.linalg.lstsq to solve for beta on the training data.
# Read the documentation for the rcond argument.

# TODO 15: create X_test in the same column order and calculate
# X_test @ beta. Store the result as test["linear_model"].

def mae(actual, predicted):
    """Return mean absolute error."""
    # TODO 16: implement the formula using a loop or NumPy operations.

def rmse(actual, predicted):
    """Return root mean squared error."""
    # TODO 17: implement the formula using a loop or NumPy operations.

models = ["persistence", "seasonal_naive", "linear_model"]
for name in models:
    # TODO 18: print the model name, MAE, and RMSE.

# TODO 19: plot actual load and all three predictions over the final
# seven days. Save the figure as "week19_predictions.png".

# TODO 20: create residual = actual - linear_model prediction.
# Plot residual against time and add a horizontal line at zero.
```

Create this results table:

Method	Information used	Test MAE	Test RMSE
Persistence	previous hour		
Seasonal naive	same hour yesterday		
Linear regression	listed lag/calendar features		

Answer:

1. Which method has the lowest MAE? Which has the lowest RMSE?
2. Does the linear model beat both baselines? Quantify the difference rather than writing only "yes" or "no."
3. Where in the final seven-day plot do the models make their largest visible errors?
4. Are residuals centered near zero? Do they show a remaining daily, weekly, or trend pattern?
5. Name one reason this synthetic result may not transfer to real smart-meter data.
6. If large peak-hour misses are more costly than ordinary misses, what additional evaluation would you perform?

Step 5: One small extension—AI permitted after your own attempt

Choose exactly one extension:

- compare errors for weekdays and weekends;
- compare peak hours (for example 4–9 p.m.) with other hours;
- add `lag_168` and determine whether a weekly lag helps;
- replace the chronological holdout with three forward-moving evaluation windows; or
- simulate a sudden level change and test which method adapts fastest.

State the extension question before changing code. Submit one figure or table and a five-sentence conclusion containing: result, evidence, limitation, possible explanation, and next test.

Part II: From Week 18 Homework to Research Ideas

Bring the completed Week 18 Research Inspiration Summary. Missing or incomplete Week 18 work should be finished before scoring ideas, because the purpose is to build from checked examples rather than generate unsupported topics.

Step 1: Extract one reusable move from each material

Complete the table in short, precise phrases. Cite a page, paragraph, figure, or instructor-provided excerpt for every source-supported statement.

Material	Problem or gap	Reusable method, measurement, or research move	Limitation or open question	Change to my thinking
1				
2				
3				

For each row, label the limitation as one of:

- **Stated:** the source explicitly reports it;
- **Inferred:** you believe it follows from the design, but the source does not directly state it; or
- **Unknown:** the available material is too short to determine it.

Step 2: Generate six distinct idea cards before judging them

Produce six ideas using this distribution:

- two **transfer** ideas: adapt a method or measurement to a justified new context;
- one **comparison** idea: compare methods, measurements, groups, or conditions fairly;
- one **hidden-assumption** idea: test what happens when an important assumption fails;
- one **enabling-resource** idea: create a small dataset, visualization, checklist, or tool and evaluate its usefulness; and
- one **wildcard** idea: follow your strongest curiosity, even if it does not use time-series analysis.

Complete one card for each idea:

Spark and source	What observation or Week 18 material produced this idea?
Stakeholder	Who might use, experience, or be affected by the result?
Testable question	What exactly could be measured, compared, or created?
Evidence available now	What data, sources, people, tools, or observations can you access immediately?
Baseline or comparison	What simple alternative prevents a meaningless result?
Smallest complete output	What honest artifact or result could be finished within three weeks?
Largest assumption	What must be checked before committing?
Boundary	What would this project not establish or claim?

If an idea uses time series, also specify the sampling frequency, prediction horizon or event window, information available at prediction time, chronological test period, and seasonal baseline. Do not add time-series modeling when time order is irrelevant.

Step 3: Score all six, then select three finalists

Score each criterion from 1 (weak) to 5 (strong). A score without a written reason does not count.

Criterion	Idea 1	Idea 2	Idea 3	Idea 4	Idea 5	Idea 6
Genuine curiosity						
Importance to a stakeholder						
Data/resource access now						
Measurable outcome and baseline						
Small enough for three weeks						
Responsible, clear boundary						
Total						

Below the table, write one evidence sentence for every score of 4 or 5 and one improvement action for every score of 1 or 2. Select three finalists. The highest total does not have to win; explain any decision that differs from the numerical ranking.

Step 4: Perform a reality check for the top two ideas

Spend 20–30 minutes on each of the top two ideas. Choose the smallest check that could change your mind:

- open the proposed data and record columns, time range, scale, missingness, and license/access;
- reproduce one baseline calculation or simple plot;
- read two methods sections to verify how a key outcome is defined;
- ask a stakeholder or knowledgeable person one precise question; or
- test whether the required software, hardware, or permissions are available.

For each idea, record:

1. what you checked and why it could change the decision;
2. the source, person, file, or tool consulted;
3. what you observed directly;
4. what remains an inference or unknown; and
5. whether the idea should continue, shrink, change, or stop.

Step 5: Proposal Seed v0.2—primary plus backup

Write one page for the primary idea and one half-page for the backup.

The **primary idea** must include:

1. working title and the personal or intellectual spark;
2. bounded problem, context, and stakeholder;
3. one research question with a measurable outcome;
4. evidence and resources confirmed so far;
5. first method and meaningful baseline;
6. smallest complete output possible within three weeks;
7. largest technical and human/stakeholder risks;
8. a clear statement of what the project will not claim; and
9. the specific lesson borrowed from one Week 18 material.

The **backup idea** must include the question, accessible evidence, baseline, minimum output, and the condition that would cause you to switch from the primary idea.

Use **Known**, **Assumption**, and **Question** labels throughout both proposals.

Step 6: Commit to one 72-hour evidence action

Complete this decision statement:

Within 72 hours, I will _____.

This will test the assumption that _____.

If I observe _____, I will continue.

If I observe _____, I will revise or switch to the backup.

The action must produce visible evidence: a file, plot, table, documented conversation, reproduced result, or checked data description.

Feynman Technique Post

Create one public-facing learning note titled **"Why You Should Not Shuffle Tomorrow into Yesterday."** Include:

- an original sketch showing a chronological training period followed by a test period;
- a plain-language explanation of lag features and data leakage;
- persistence and seasonal-naive forecasting in one sentence each;
- the final seven-day prediction plot;
- one example showing why an average error can hide an important failure; and
- one sentence connecting a model's unexplained error to a possible research idea.

Submission checklist. The time-series code runs from start to finish; all manual-core `TODO`s were completed without AI; features use no future target information; the split is chronological; three methods are compared with manually implemented MAE and RMSE; figures have labeled axes; residuals are interpreted; six genuinely different ideas were generated before scoring; two ideas received real evidence checks; the primary and backup proposals distinguish known facts, assumptions, and questions; the 72-hour action has a decision rule; sources and AI assistance are disclosed.